CS315 · ALGORITHMS ANALYSIS & DESIGN

Midterm 462 Solutions

25 marks

Muharib

01 Solution Q1: linear probing

Key	key mod 7	Probes	Cell
76	6	6	**6**
93	2	2	**2**
40	5	5	**5**
47	5	5 → 6 → 0	**0**
10	3	3	**3**
55	6	6 → 0 → 1	**1**

0	1	2	3	4	5	6
47	55	93	10		40	76

```
47:  h = 47 mod 7 = 5 taken   →   6 taken   →   (6 + 1) mod 7 = 0
```

```
55:  h = 55 mod 7 = 6 taken   →   0 taken   →   1
```

Note the wrap-around from the end of the table (6) to its start (0).

02 Solution Q2: True / False + MCQ

#	Answer	Reason
1	**a. True**	A character that occurs more often gets a shorter code.
2	**b. False**	Greedy does not give the optimum for every DP problem.
3	**c**	Dynamic programming stores the results of sub-problems and reuses them.
4	**b. False**	The correct equation ends with `T(MERGE)`, not $T(\frac{MERGE}{2})$.
5	**c. Other value**	The time is `O(n)` when the activities are sorted by finish time.
6	**a. True**	DP solves each sub-problem once and stores its result, which saves a lot of time.
7	**d**	Little-omega means strictly greater. Greater than or equal is big-Omega.
8	**C. O(n²)**	Two nested loops, each running `n` times.
9	**A. O(1)**	The best case: the element is in the first cell.
10	**b**	Binary search halves the range every time.

03 Solution Q3: greedy knapsack

Greedy sorts the items by value per kilogram (value ÷ weight) from largest to smallest and

takes them in that order until the bag is full.

Item	Weight (kg)	Value (SAR)	Value per kg	Taken
Saffron	2	4000	2000	2 kg → 4000
Cardamom	2	1000	500	2 kg → 1000
Cinnamon	5	500	100	1 kg → 100
Pepper	10	500	50	0

```
Weight used = 2 + 2 + 1 = 5 kg
```

```
Total value = 4000 + 1000 + 100 = 5100 SAR
```

Where is the problem? If items must be taken whole (0-1 knapsack), greedy takes saffron and cardamom and stops at 4 kg and SAR 5000, leaving one kilogram it cannot fill. The fix is to allow taking part of an item (fractional knapsack), so we take one kilogram of cinnamon and the total becomes 5100.

04 Solution Q4: LCS

```
X[i] = Y[j]:   c[i, j] = c[i−1, j−1] + 1        diagonal + 1
```

```
X[i] ≠ Y[j]:   c[i, j] = max(top, left)
```

	⌀	A	B	C	A	D	F
⌀	0	0	0	0	0	0	0
A	0	**1**	1	1	1	1	1
C	0	1	1	**2**	2	2	2
B	0	1	2	2	2	2	2
A	0	1	2	2	**3**	3	3
E	0	1	2	2	3	3	3
D	0	1	2	2	3	**4**	4

```
Length = 4
```

```
LCS = ACAD        (ABAD is also a correct answer)
```

The coloured cells are the traceback path from the last cell: at every match we take the letter and move diagonally.